

```
-----  
-----  
-- Engineer: Olasubomi Borishade  
--  
-- Create Date: 06/04/2026 01:42:44 AM  
-- Design Name: Systolic Array Test Bench  
-- Module Name: Systolic_Array_Test_Bench - Behavioral  
-- Project Name: Parameterized Matrix Multiply Engine  
  
-- Description: A test bench for a grid of interconnected  
processing units  
-- that pass data to one another in a pipelined fashion.  
  
-- Revision:  
-- Revision 0.01 - File Created  
-- Additional Comments:  
--  
-----  
-----
```

```
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
use IEEE.NUMERIC_STD.ALL;  
use WORK.ARRAY_CUSTOM_PACK.ALL;  
use STD.ENV.FINISH;
```

```
-- Uncomment the following library declaration if instantiating  
-- any Xilinx leaf cells in this code.
```

```
--library UNISIM;  
--use UNISIM.VComponents.all;
```

```
entity Systolic_Array_Test_Bench is  
-- Port ( );  
end Systolic_Array_Test_Bench;
```

```
architecture Behavioral of Systolic_Array_Test_Bench is
```

```

Constant Rows          : INTEGER := 2;
Constant Cols          : INTEGER := 2;
Constant Input_Widths  : INTEGER := 8;
Constant Accumulator_Widths : INTEGER := 32;
Constant Clock_Period  : TIME     := 10 ns;

Signal Input_As         : SignedArray_1d(0 to Rows -
1)(Input_Widths - 1 downto 0) := (others => (others => '0'));
Signal Input_Bs         : SignedArray_1d(0 to Cols -
1)(Input_Widths - 1 downto 0) := (others => (others => '0'));
Signal Clocks           : STD_LOGIC := '0';
Signal Resets           : STD_LOGIC := '0';
Signal Enables          : STD_LOGIC := '0';
Signal Clear_Accumulators : STD_LOGIC := '0';
Signal Outputs          : SignedArray_2d(0 to Rows - 1)(0 to
Cols - 1)(Accumulator_Widths - 1 downto 0);

```

```
begin
```

```
Systolic_Array_Inst: entity work.Systolic_Array(Behavioral)
```

```

    Generic map (    Rows          => Rows,
                    Cols          => Cols,
                    Input_Width  => Input_Widths,
                    Accumulator_Width => Accumulator_Widths
                    )

```

```

    Port map      (    Input_A      => Input_As,
                    Input_B      => Input_Bs,
                    Clock        => Clocks,
                    Reset        => Resets,
                    Clear_Accumulator => Clear_Accumulators,
                    Enable       => Enables,
                    Output       => Outputs
                    );

```

```
Clock_Process : process
```



```

Input_As(0) <= to_signed(0, Input_Widths);
Input_As(1) <= to_signed(0, Input_Widths);
Input_Bs(0) <= to_signed(0, Input_Widths);
Input_Bs(1) <= to_signed(0, Input_Widths);
wait for 2 * Clock_Period;

Enables      <= '0';
wait for 2 * Clock_Period;

assert (Outputs(0)(0) = to_signed(19, Accumulator_Widths))
    report "TEST 1a FAILED: (0,0) expected 19, got " &
integer'image(to_integer(Outputs(0)(0)))
    severity error;

assert (Outputs(0)(1) = to_signed(22, Accumulator_Widths))
    report "TEST 1b FAILED: (0,1) expected 22, got " &
integer'image(to_integer(Outputs(0)(1)))
    severity error;

assert (Outputs(1)(0) = to_signed(43, Accumulator_Widths))
    report "TEST 1c FAILED: (1,0) expected 43, got " &
integer'image(to_integer(Outputs(1)(0)))
    severity error;

assert (Outputs(1)(1) = to_signed(50, Accumulator_Widths))
    report "TEST 1d FAILED: (1,1) expected 50, got " &
integer'image(to_integer(Outputs(1)(1)))
    severity error;

```

```

-----
-----

```

-----  
-----  
Clear\_Accumulators <= '1';  
wait for Clock\_Period;

Clear\_Accumulators <= '0';  
Enables <= '1';  
Input\_As(0) <= to\_signed(-1, Input\_Widths);  
Input\_As(1) <= to\_signed(-3, Input\_Widths);  
Input\_Bs(0) <= to\_signed(-5, Input\_Widths);  
Input\_Bs(1) <= to\_signed(-6, Input\_Widths);  
wait for Clock\_Period;

Input\_As(0) <= to\_signed(-2, Input\_Widths);  
Input\_As(1) <= to\_signed(-4, Input\_Widths);  
Input\_Bs(0) <= to\_signed(-7, Input\_Widths);  
Input\_Bs(1) <= to\_signed(-8, Input\_Widths);  
wait for Clock\_Period;

Input\_As(0) <= to\_signed(0, Input\_Widths);  
Input\_As(1) <= to\_signed(0, Input\_Widths);  
Input\_Bs(0) <= to\_signed(0, Input\_Widths);  
Input\_Bs(1) <= to\_signed(0, Input\_Widths);  
wait for 2 \* Clock\_Period;

Enables <= '0';  
wait for 2 \* Clock\_Period;

assert (Outputs(0)(0) = to\_signed(19, Accumulator\_Widths))  
report "TEST 2a FAILED: (0,0) expected 19, got " &  
integer'image(to\_integer(Outputs(0)(0)))  
severity error;

assert (Outputs(0)(1) = to\_signed(22, Accumulator\_Widths))  
report "TEST 2b FAILED: (0,1) expected 22, got " &

